

Arseniy Buzyan

Hela Kedjan

Automationsparadigmet: Varför bygga en kedja?

Fragmenterad IT / Utan kedja

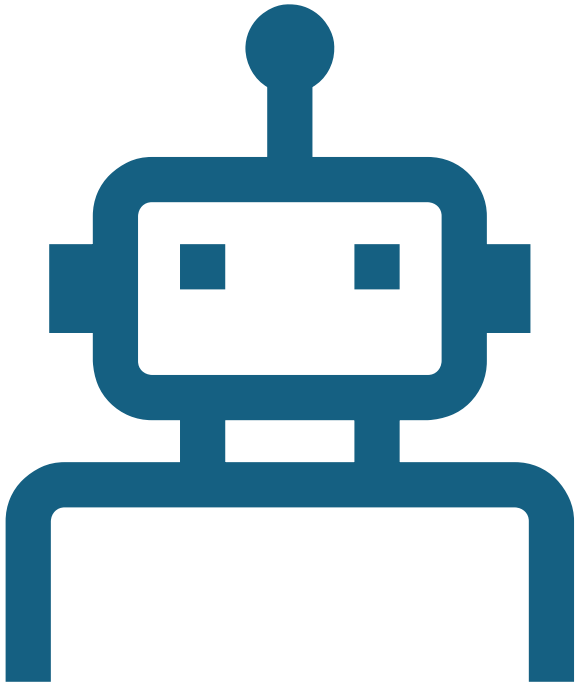
- Övergångar: Manuella steg mellan verktygen.
- Riskprofil: Hög risk för mänskliga fel.
- Drift: Svårt att återskapa miljöer.

Kontinuerlig Automation / Med kedja



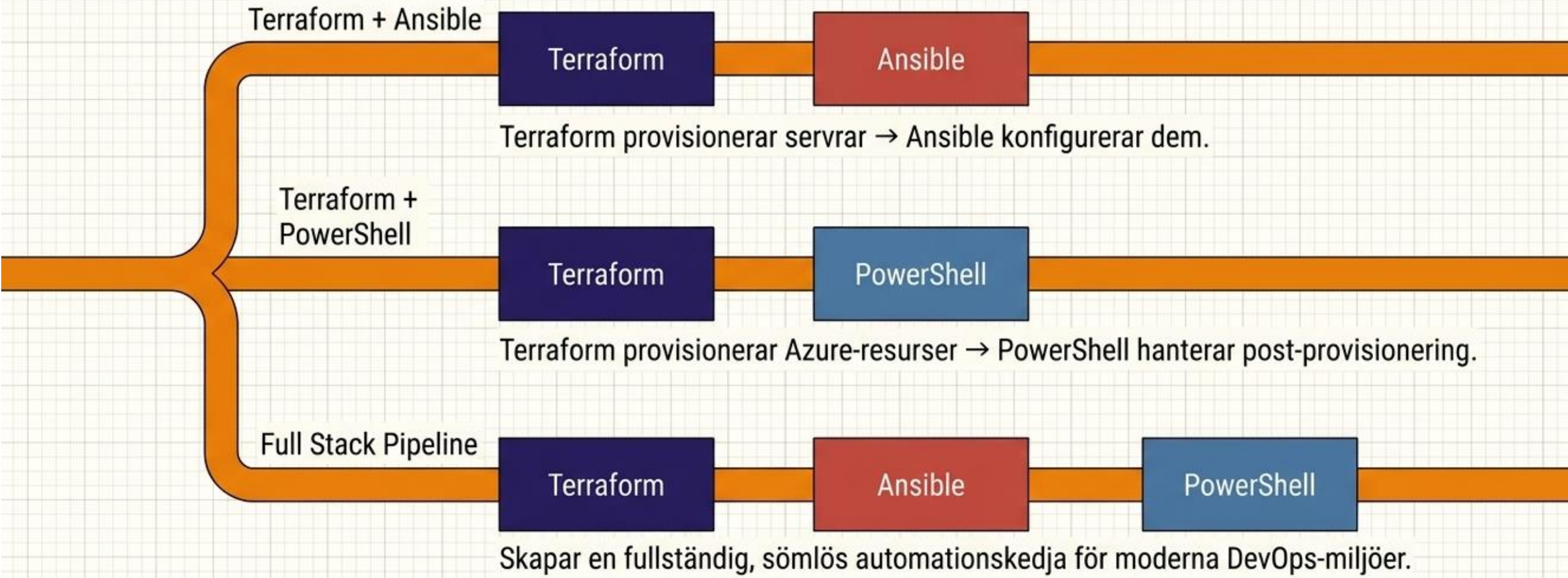
- Övergångar: Automatisk övergång (Build → Configure → Control).
- Riskprofil: Konsekvent och repeterbar process.
- Drift: Fullständig historik och spårbarhet.

Från Verkligheten – Så Här Används Kedjor i Företag



- I riktiga IT-miljöer är det sällan bara ETT verktyg – det är KEDJAN som gör jobbet.
- Varför detta är viktigt: Enligt en teknisk guide från Azure Local kan Terraform och Ansible köras i tre mönster: Terraform + PowerShell, Terraform + Ansible, eller Ansible-only – beroende på vad som passar bäst för din miljö . I många moderna DevOps-miljöer används alla tre för att täcka olika aspekter av automationsbehovet.
- Varför detta är viktigt: Enligt en tekniskt artikel om automation är integrationen av skriptspråk, konfigurationshantering och Infrastructure as Code nyckeln till att uppnå sömlös automatisering i moderna IT-miljöer.

Industriell Standard: Kedjor i Verkligheten



De Tre Rollerna i Automationsfabriken

1. Provisionera

Verktyg: Terraform



- **Uppgift:** Skapar servrar, nätverk, lagring.
- **Styrka:** Bäst på infrastruktur – idempotent och deklarativt.

2. Konfigurera

Verktyg: Ansible



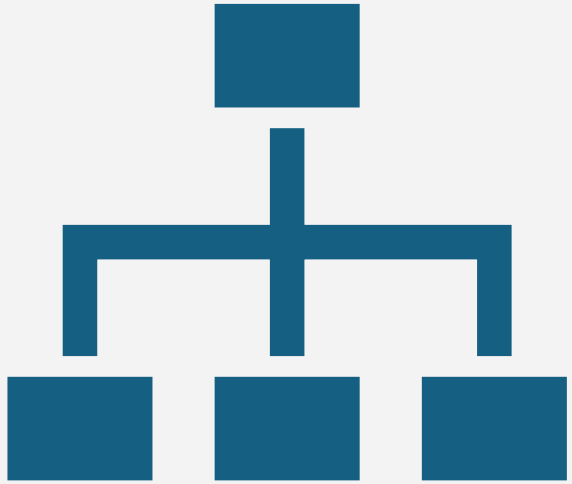
- **Uppgift:** Installerar och konfigurerar mjukvara.
- **Styrka:** Agentlöst, YAML-baserat, idempotent.

3. Verifiera

Verktyg: PowerShell / Python



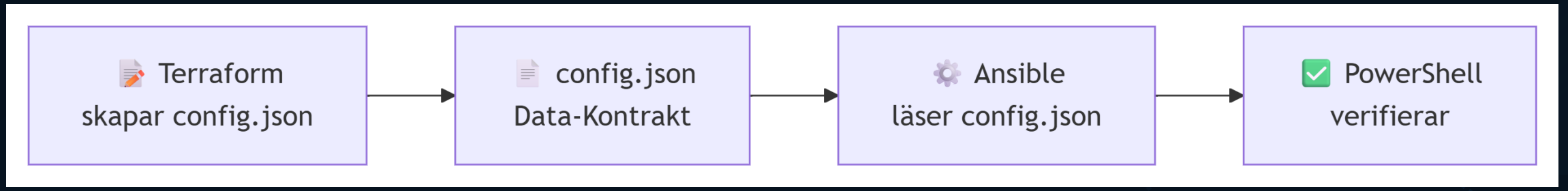
- **Uppgift:** Kontrollerar att allt fungerar.
- **Styrka:** Operativ kontroll och slutgiltig orkestrering.



Vad varje steg gör

Enligt Microsofts dokumentation: Terraform, Ansible, Chef, Puppet, PowerShell och CLI är alla verktyg som kan användas för att automatisera provisionering och konfiguration, men rekommendationen är att använda ett deklarativt IaC-verktyg som Terraform för provisionering och ett konfigurationsverktyg som Ansible för installation .

Steg	Verktyg	Vad gör det?	Varför just detta verktyg?
1. Provisionera	Terraform	Skapar servrar, nätverk, lagring	Bäst på att skapa infrastruktur – idempotent och deklarativt
2. Konfigurera	Ansible	Installerar och konfigurerar mjukvara	Bäst på konfiguration – agentlöst, YAML-baserat, idempotent
3. Verifiera	PowerShell/Python	Kontrollerar att allt fungerar	PowerShell för Windows, Python för orkestrering och rapportering



Dataflöde – JSON som "Kontrakt" Mellan Stegen

Ett enkelt JSON-dokument (konfigurationsfil) är "limmet" som håller ihop kedjan.

Vad config.json innehåller

Varför JSON? JSON är standardformatet för konfiguration i moderna IT-system . Enligt en teknisk artikel om PowerShell-automation med Terraform och Ansible används JSON-filer ofta som en "single source of truth" – en enda källa som alla verktyg läser från för att undvika konflikter . I många Enterprise-miljöer används en gemensam konfigurationskälla (`variables.yml`) som alla verktyg konsumerar, vilket säkerställer att infrastrukturen blir identisk oavsett vilket verktyg som använder den .

Fält	Exempelvärde	Användning
server_name	"WEB01"	Används av Ansible för att skapa webbsida
port	80	Används av Ansible och PowerShell för verifiering
environment	"production"	Används av alla steg för att veta vilken miljö som gäller
log_path	"C:/Projekt/Logs"	Används av PowerShell för loggning

Versionshantering (Git) – Glöm inte Detta!

Utan Git har du ingen historik över dina automationsskript. I produktion är det oacceptabelt!



Varför Git är viktigt för din kedja

Anledning	Exempel
Historik	Vem ändrade vad och när?
Återställning	Om en ändring bryter kedjan – återställ till tidigare version
Samarbete	Flera personer kan jobba på samma kedja
Spårbarhet	Koppla ändringar i kedjan till ärenden (t.ex. Jira)

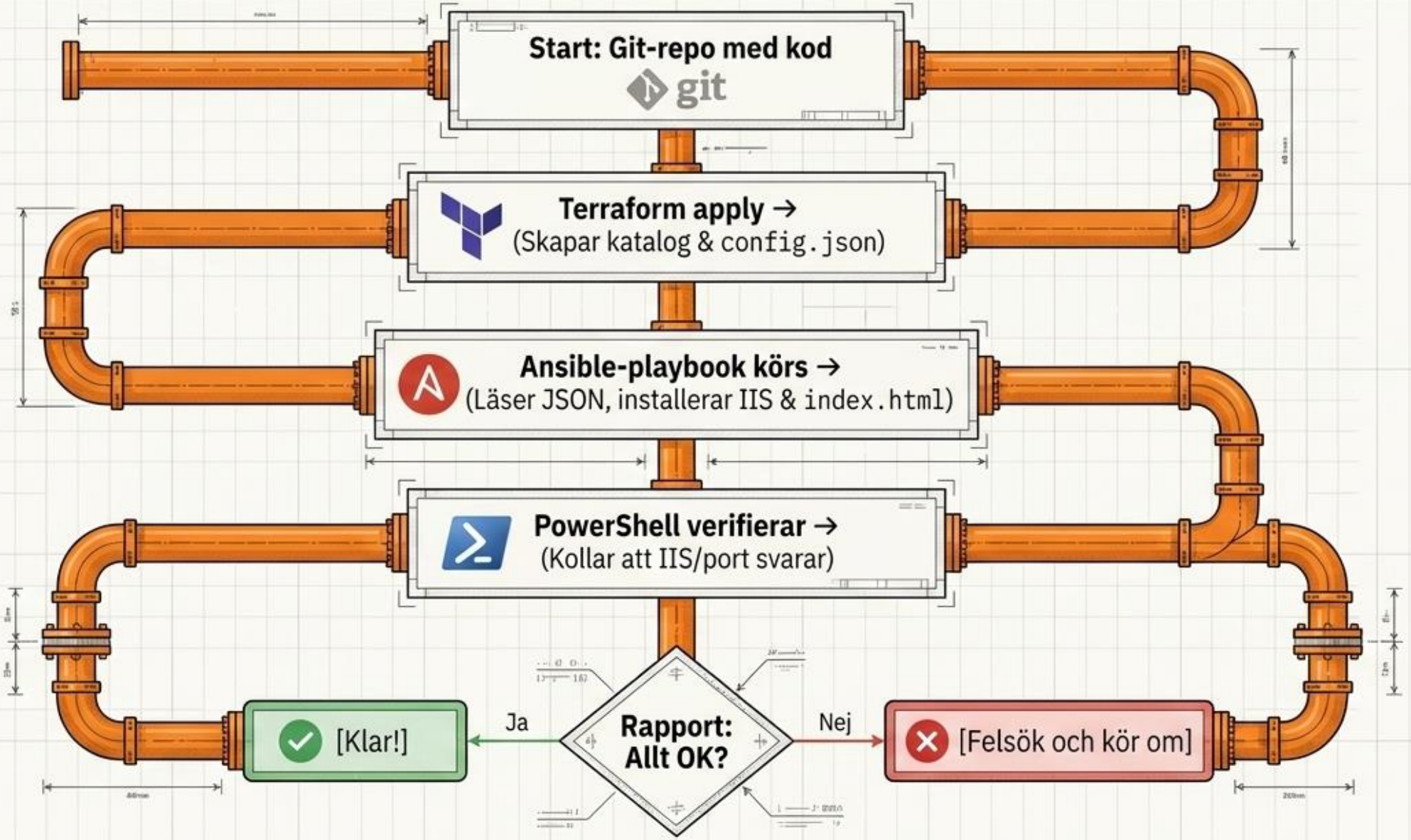
Grundläggande Git-kommandon för din kedja

Varför detta är viktigt: Enligt en bok om DevOps-automation är Git (versionshantering) och CI/CD-pipelines centrala för att bygga repeterbara och pålitliga automationskedjor . I en avancerad DevOps-miljö ska du kunna spåra varje ändring och i värsta fall återställa hela automationskedjan med ett enda Git-kommando.



```bash
git init                          # Starta ett nytt repository
git add .                         # Lägg till alla filer
git commit -m "Lade till Terraform-konfiguration"  # Spara version
git push origin main              # Ladda upp till GitHub
```

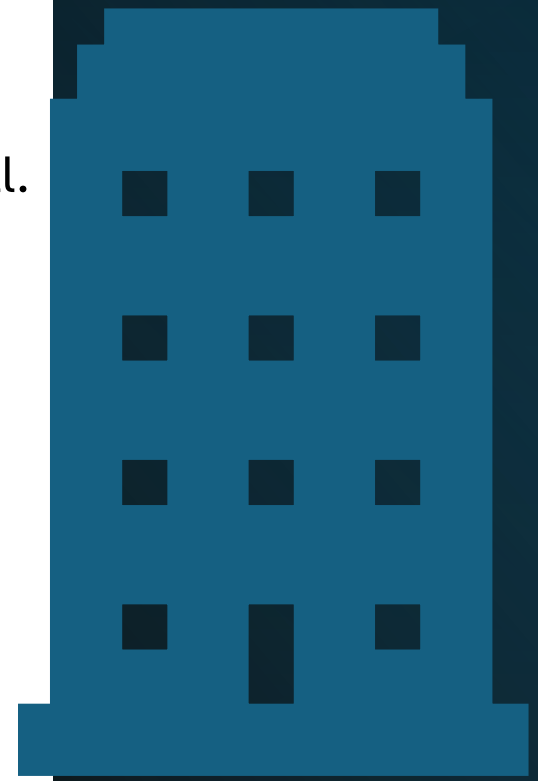
Hela Kedjan: Master Blueprint



Steg-för-steg beskrivning

Enligt Microsofts dokumentation: Terraform, Ansible, PowerShell och CLI kan alla användas för att bygga en komplett automationskedja. Nyckeln är att varje verktyg gör det det är bäst på: Terraform för provisionering, Ansible för konfiguration, PowerShell för operativ kontroll.

Steg	Vad händer?	Resultat
1. Terraform	Skapar katalogstruktur och config.json med inställningar	C:/Projekt/Config/config.json
2. Ansible	Läser config.json, installerar IIS och skapar webbsida	IIS installerat, index.html skapad
3. PowerShell	Kontrollerar att IIS är igång och att webbsidan svarar	Loggfil med resultat
4. Python	Orkestrerar alla steg (valfritt – för VG)	Slutrapport skapad



Workshop – Bygg Din Första Kedja

Mål: Skapa en komplett kedja från Terraform → Ansible → PowerShell.



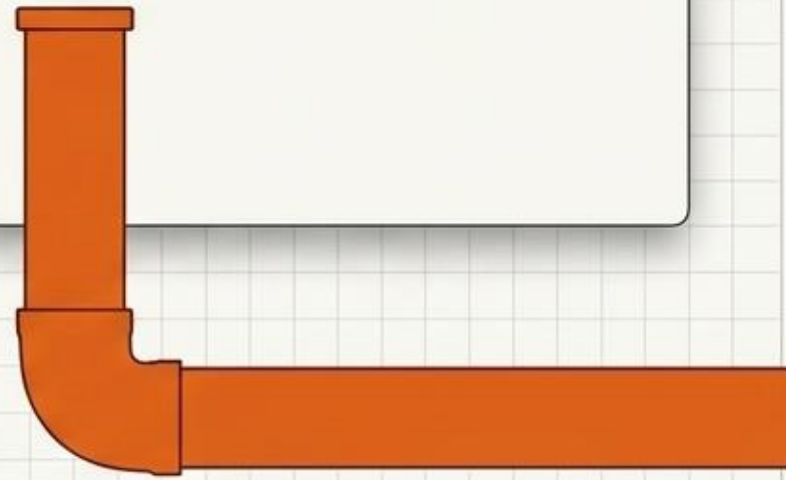
Steg 1: Terraform Provisionerar (Skapar Kontraktet)

Mål: Skapa en komplett kedja från Terraform → Ansible → PowerShell.

Terraform initierar miljön och genererar config.json som kommer att styra resten av maskineriet.

main.tf

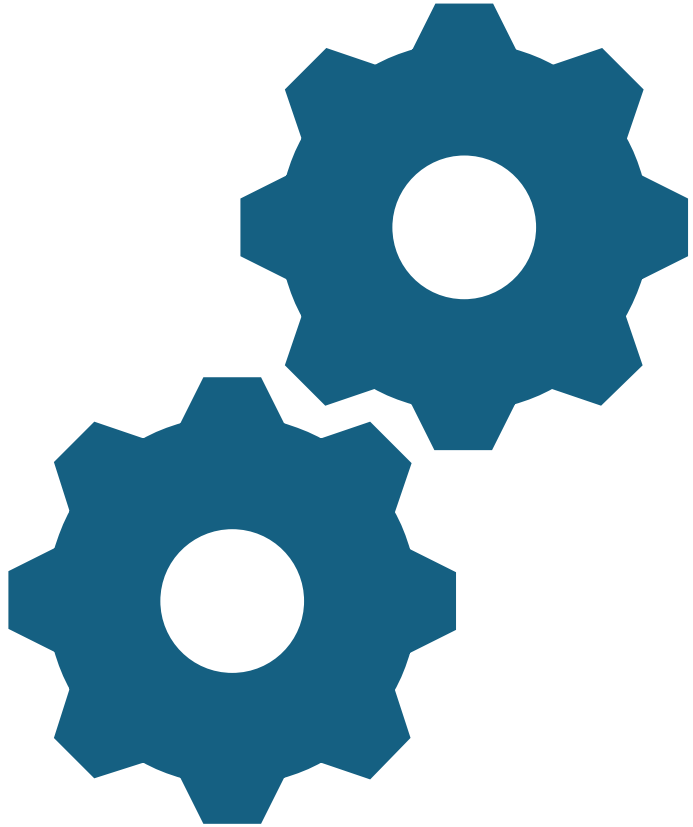
```
resource "local_file" "config" {
  filename = "C:/Projekt/Config/config.json"
  content = jsonencode({
    server_name = "WEB01"
    port = 80
  })
}
```



Steg 1 – Terraform

hcl

```hcl
# main.tf - skapa config.json
resource "local_file" "config" {
  filename = "C:/Projekt/Config/config.json"
  content = jsonencode({
    server_name = "WEB01"
    port = 80
  })
}
```



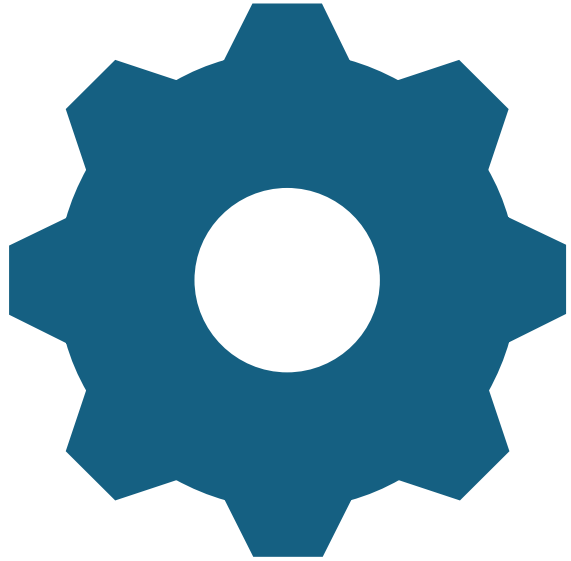
Steg 2: Ansible Konfigurerar (Läser Kontraktet)

Nu tar Ansible över. Istället för hårdkodade värden läser Ansible dynamiskt den config.json som Terraform just skapade.

```
# playbook.yml

- hosts: localhost
  tasks:
    - name: Läs config.json
      set_fact:
        config: "{{ lookup('file',
'C:/Projekt/Config/config.json') | from_json }}"
```

Steg 2 – Ansible



yaml

```yaml
# playbook.yml – läs config.json och konfigurera
- hosts: localhost
  tasks:
    - name: Läs config.json
      set_fact:
        config: "{{ lookup('file', 'C:/Projekt/Config/config.json') | from_json }}"
```

Steg 3: PowerShell Verifierar (Bekräftar Kontraktet)

Sista steget. PowerShell agerar kvalitetskontroll och säkerställer att webbservern faktiskt svarar på den port som definierades i startpunkten.

Detta är arkitekturen för ditt slutprojekt.

```
# verify.ps1

$config = Get-Content
"C:/Projekt/Config/config.json" |
ConvertFrom-Json

$response = Invoke-WebRequest -Uri
"http://localhost:$($config.port)"
```



Steg 3 – PowerShell

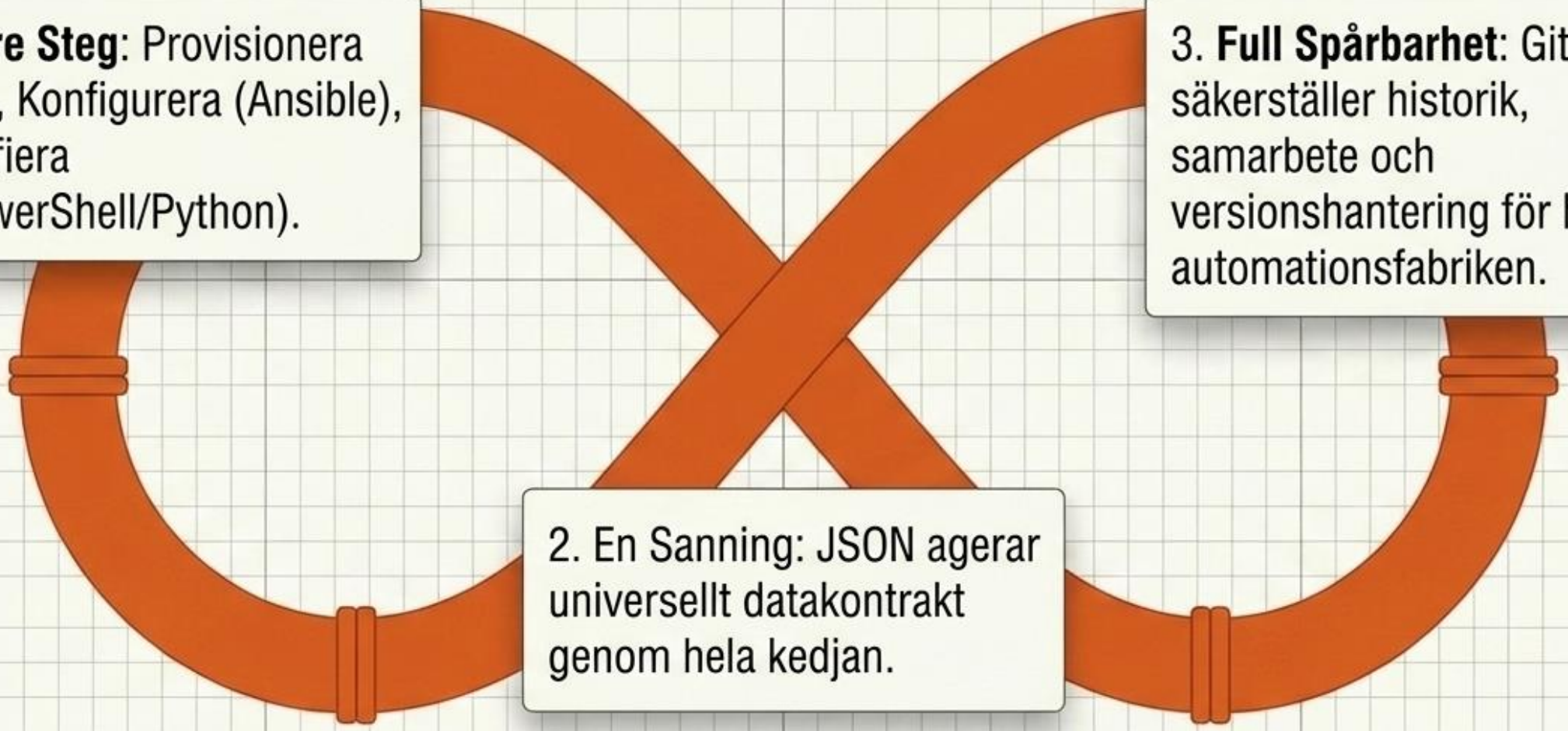
powershell

```powershell
# verify.ps1 – verifiera att servern svarar
$config = Get-Content "C:/Projekt/Config/config.json" | ConvertFrom-Json
$response = Invoke-WebRequest -Uri "http://localhost:$($config.port)"
```

Koppling till examinationen: Detta är exakt den kedja du kommer att bygga i ditt slutprojekt!

Sammanfattning: Ett Kontinuerligt Flöde

1. **Tre Steg**: Provisionera (TF), Konfigurera (Ansible), Verifiera (PowerShell/Python).



2. En Sanning: JSON agerar universellt datakontrakt genom hela kedjan.

3. **Full Spårbarhet**: Git säkerställer historik, samarbete och versionshantering för hela automationsfabriken.

Sammanfattning

Kedjan består av tre steg:

Kedjan består av tre steg:

1. Provisionera med Terraform – skapa infrastruktur

2. Konfigurera med Ansible – installera och ställa in mjukvara

3. Verifiera med PowerShell/Python – kontrollera att allt fungerar

JSON som "kontrakt": Terraform skapar JSON → Ansible och PowerShell läser JSON

Git är viktigt: Versionshantering av hela kedjan, spårbarhet och samarbete

Idempotent: Varje steg i kedjan ska tåla att köras om och om igen

Quiz



https://forms.gle/xfpeDiMVWNRtJVGa6